

WEB INFRASTRUCTURE

TECHNICAL INFRASTRUCTURE REPORT

Building and Securing a Mini Production-Ready Web Infrastructure

Assessment Details	
Module	Web Infrastructure (BSc(Hons) Software Engineering, Year 1)
Faculty / Facilitator	POHUR Navindranath Sharma
Submission date	8th September 2026
Environment used	Ubuntu 26.04 LTS running under WSL2 (Windows Subsystem for Linux), kernel 6.18.33.2-microsoft-standard-WSL2
Team Webserver	• Edith Amondi • Rebecca Adhiambo • Leatitia Mizero • Nelson Fodjo Kamdoun
Technical Work Repo	https://github.com/Becca4375/WEBSERVER.git

Table of Contents

Table of Contents.....	2
1. Architecture.....	4
1.1 Architecture Diagram.....	4
2. Linux Environment (Part A).....	6
2.1 Identifying the Environment.....	6
2.2 Creating the Second User Account.....	7
3. SSH: Client and Server on the Same Machine (Part B).....	8
3.1 Installing and Starting the SSH Server.....	8
3.2 Inspecting sshd_config Settings.....	9
3.3 Password-Based Authentication& Failed Attempt Logging.....	11
3.4 Deliberately failed login.....	12
3.5 Key-Based Authentication & Key Pair Deployment.....	13
3.6 File Transfer over SSH(SCP and SFTP).....	17
4. NGINX Web Server (Part C).....	19
4.1 Installing and Verifying NGINX.....	19
4.2 NGINX Directory Hierarchy and Symlink Architecture.....	20
4.3 Custom Website Creation.....	21
4.4 Server Block Configuration & Syntax validation.....	22
4.5 Validating and Accessing the Website.....	24
5. HTTP Methods, Status Codes, and Custom Headers (Part D).....	26
5.1 Testing HTTP Methods (GET, POST, PUT, DELETE, HEAD).....	26
5.2 Deliberate Status Code Generation (200, 403, 404, 405).....	27
6. Telnet, Ports and Raw HTTP (Part E).....	31
6.1 Port Reachability testing (ports 22 and 80).....	31
6.2 Service State Behavior: Running vs. Stopped NGINX.....	32
6.3 Raw HTTP Request Construction via Telnet/Netcat.....	34
Repeat 1: request for a page that does not exist.....	36
Repeat 2: malformed request (missing Host header).....	37
7. API Interaction (Part F).....	39
7.1 Public API Overview (JSONPlaceholder).....	39
7.2 Executing Three Distinct API Requests.....	39
7.3 Error Generation, Diagnostics, and Correction.....	42
8. Environment Troubleshooting Scenario (Part G).....	44
8.1 Stage 1: Problem and Symptom.....	44
8.2 Stage 2: Log and System Evidence.....	44
8.3 Stage 3: Technical Investigation.....	45
8.4 Stage 4: Root Cause Analysis.....	46
8.5 Stage 5: Remediation and Verification.....	46
9. Conclusion.....	48
10. References & Submission Directory Structure.....	49
10.1 Sources, Credits & AI Transparency Log.....	50
10.2 Link to submission folder.....	50

1. Architecture

For the whole practice, we used one computer and installed WSL2, which gave us a real Linux system inside Windows. This allowed us to meet the assignment instructions of using one Linux laptop, where we used Ubuntu 26.04. And as result, no second computer, online servers, or any other intermediaries were needed between the host and VM.

However, using one computer does not remove the fact that we had to show client and server interaction, or talking to each other. So, to demonstrate that, we had to create two different users on the same Linux system and are as follows.

- **becca**: This is the default user. We used it to install and manage concepts like SSH and NGINX.
Additionally, performing as a client to send requests using commands like curl, ssh, etc.
- **webserver**: This was created for the sake of testing SSH and acted as a server.

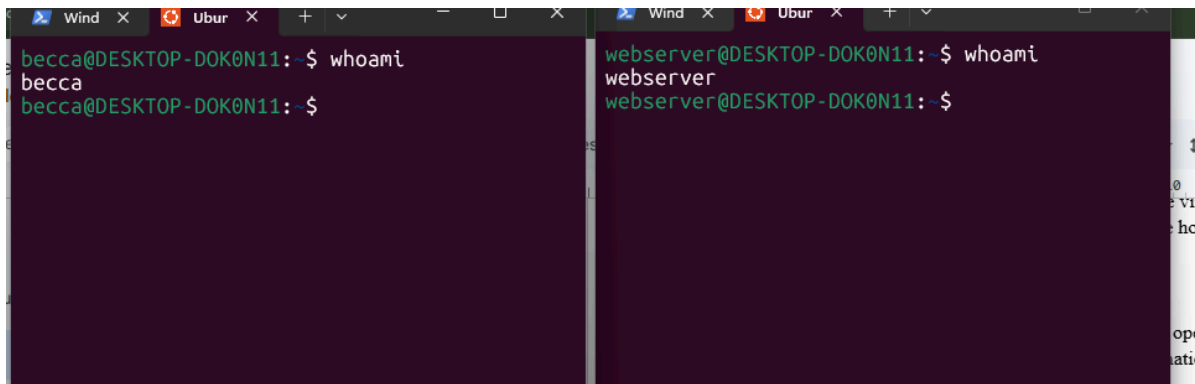
Ultimately, if the first user(becca) tries to connect to the second user (webserver, acting like a server) using SSH, it is like one computer talking to itself but using two different identities. And this proves that login, file ownership, and permissions really work, even with just one computer.

1.1 Architecture Diagram

```
UBUNTU 26.04 LTS (WSL2) , single Linux kernel, single machine
|
+-- SERVER ROLES (services listening for requests)
|   +-- OpenSSH Server ----- 0.0.0.0:22  (ssh)
|   +-- NGINX Web Server ----- 0.0.0.0:80  (nginx)
|   +-- Website files ----- /var/www/teamwebserver/
|
+-- USER ACCOUNTS (on the same filesystem)
|   +-- becca      -> primary login, sudo/admin rights
|   +-- webserver  -> second account, SSH test target,
|                       owns /home/webserver and its own authorized_keys
|
+-- CLIENT ROLES (programs that send requests, run as becca)
    +-- SSH client ----- ssh webserver@localhost
```

```
+-- Telnet client ----- telnet localhost 22 / telnet localhost 80
+-- curl / Browser ---- http://localhost
+-- curl / Postman ---- https://api.publicapis.example (public API)
```

The client-server architecture depicted in the diagram above shows that they are just roles played by terminal sessions, not separate physical computers. Moreover, NGINX and sshd are running in the background at all times. They open a port and wait for connections through a listening process. But they are different from curl, ssh, and telnet, because these run for a short time by connecting, sending a request, getting a response, and then closing.



```
becca@DESKTOP-DOK0N11: $ whoami
becca
becca@DESKTOP-DOK0N11: $

webserver@DESKTOP-DOK0N11: $ whoami
webserver
webserver@DESKTOP-DOK0N11: $
```

What this proves: The left terminal represents the client computer (becca), and the right terminal represents the server computer(webserver).

2. Linux Environment (Part A)

2.1 Identifying the Environment

The following commands were run as becca to record the exact environment used and screenshot showing the results:

```
$ hostname
$ whoami
$ hostname -I
$ ip addr
$ cat /etc/os-release
```

```
becca@DESKTOP-DOK0N11: $ hostname
DESKTOP-DOK0N11
becca@DESKTOP-DOK0N11: $ whoami
becca
becca@DESKTOP-DOK0N11: $ hostname -I
192.168.199.170
becca@DESKTOP-DOK0N11: $ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet 10.255.255.254/32 brd 10.255.255.254 scope global lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host proto kernel_lo
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc mq state UP group default qlen 1000
    link/ether 00:15:5d:2f:65:26 brd ff:ff:ff:ff:ff:ff
    altname enx00155d2f6526
    inet 192.168.199.170/20 brd 192.168.207.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::215:5dff:fe2f:6526/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
becca@DESKTOP-DOK0N11: $ cat /etc/os-release
PRETTY_NAME="Ubuntu 26.04 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04 (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=resolute
LOGO=ubuntu-logo
```

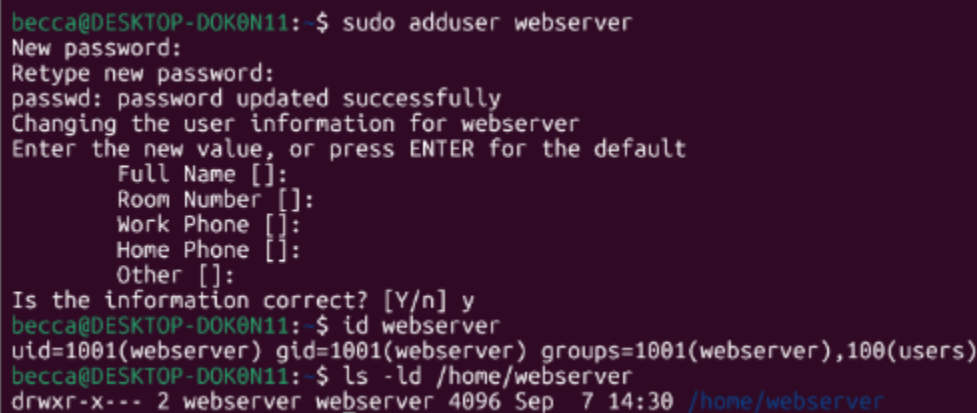
The above diagram confirms that we are operating under Ubuntu 26.0 LTS; the current user is becca, and it records the computer's hostname and IP addresses. Since the client and server programs run on the same computer and the same network, 127.0.0.1 is shown as localhost.

2.2 Creating the Second User Account

```
$ sudo adduser webserver
$ id webserver
$ ls -ld /home/webserver
```

What `sudo adduser webserver` creates:

- A new line in **/etc/passwd** with the username, an ID number, its home folder, and its default shell.
- A new line in **/etc/shadow** that stores the user's (encrypted) password.
- A new group called **webserver** in **/etc/group**; on Ubuntu, every new user gets their own group.
- A home folder at **/home/webserver**, copied from a template folder, owned by the new user, with permissions set to 750 or 755 depending on the system's defaults.



```
becca@DESKTOP-DOK0N11:~$ sudo adduser webserver
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for webserver
Enter the new value, or press ENTER for the default
  Full Name []:
  Room Number []:
  Work Phone []:
  Home Phone []:
  Other []:
Is the information correct? [Y/n] y
becca@DESKTOP-DOK0N11:~$ id webserver
uid=1001(webserver) gid=1001(webserver) groups=1001(webserver),100(users)
becca@DESKTOP-DOK0N11:~$ ls -ld /home/webserver
drwxr-x--- 2 webserver webserver 4096 Sep  7 14:30 /home/webserver
```

The above screenshot provides direct evidence that the second account exists as a real, separate identity on the filesystem, with its own home directory.

3. SSH: Client and Server on the Same Machine (Part B)

3.1 Installing and Starting the SSH Server

```
$ sudo apt update
$ sudo apt install openssh-server
$ sudo systemctl status ssh
```

```
becca@DESKTOP-DOK0N11: $ sudo apt update && sudo apt install openssh-server
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://security.ubuntu.com/ubuntu resolute-security InRelease
Hit:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
153 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
  openssh-server

Installing dependencies:
  libwrap0 openssh-sftp-server

Suggested packages:
  molly-guard monkeysphere ssh-askpass ufw

Summary:
  Upgrading: 0, Installing: 3, Removing: 0, Not Upgrading: 153
  Download size: 708 kB
  Space needed: 3116 kB / 1024 GB available

Continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu resolute/main amd64 libwrap0 amd64 7.6.q-36build2 [48.5 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 openssh-sftp-server amd64 1:10.2p1-2ubuntu3.6 [37.4 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 openssh-server amd64 1:10.2p1-2ubuntu3.6 [622 kB]
Fetched 708 kB in 4s (185 kB/s)
Preconfiguring packages ...
Selecting previously unselected package libwrap0:amd64.
(Reading database ... 39235 files and directories currently installed.)
Preparing to unpack .../libwrap0_7.6.q-36build2_amd64.deb ...
Unpacking libwrap0:amd64 (7.6.q-36build2) ...
Selecting previously unselected package openssh-sftp-server.
Preparing to unpack .../openssh-sftp-server_1%3a10.2p1-2ubuntu3.6_amd64.deb ...
Unpacking openssh-sftp-server (1:10.2p1-2ubuntu3.6) ...
Selecting previously unselected package openssh-server.
Preparing to unpack .../openssh-server_1%3a10.2p1-2ubuntu3.6_amd64.deb ...
Unpacking openssh-server (1:10.2p1-2ubuntu3.6) ...
Setting up openssh-sftp-server (1:10.2p1-2ubuntu3.6) ...
Setting up libwrap0:amd64 (7.6.q-36build2) ...
Setting up openssh-server (1:10.2p1-2ubuntu3.6) ...
Creating config file /etc/ssh/sshd_config with new version
Creating SSH2 RSA key; this may take some time ...
3072 SHA256:jKtzroFE6UIPI2t7EHEwsy1UNc170oMchqM40N08ebc root@DESKTOP-DOK0N11 (RSA)
Creating SSH2 ECDSA key; this may take some time ...
256 SHA256:46erwBEG6nynyUHuujZVzbmSoAuBH4ZfK1LcdFudsUQ root@DESKTOP-DOK0N11 (ECDSA)
Creating SSH2 ED25519 key; this may take some time ...
256 SHA256:otUXgzOIRKLI1lNkLZp0eU96kk/kqgS5wftB7dLT2To root@DESKTOP-DOK0N11 (ED25519)
Creating user 'ssh' (ssh user) with UID 986 and GID 65534.
Created symlink '/etc/systemd/system/sockets.target.wants/ssh.socket' -> '/usr/lib/systemd/system/ssh.socket'.
Created symlink '/etc/systemd/system/ssh.service.requires/ssh.socket' -> '/usr/lib/systemd/system/ssh.socket'.
Created symlink '/etc/systemd/system/ssh.service.wants/sshd-keygen.service' -> '/usr/lib/systemd/system/sshd-keygen.service'.
Created symlink '/etc/systemd/system/sshd.service.wants/sshd-keygen.service' -> '/usr/lib/systemd/system/sshd-keygen.service'.
Created symlink '/etc/systemd/system/sshd@.service.wants/sshd-keygen.service' -> '/usr/lib/systemd/system/sshd-keygen.service'.
Created symlink '/etc/systemd/system/ssh.socket.wants/sshd-keygen.service' -> '/usr/lib/systemd/system/sshd-keygen.service'.
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for libc-bin (2.43-2ubuntu2) ...
```

The above screen confirms that SSH is installed and actively listening, which is a prerequisite for every later SSH task.

3.2 Inspecting sshd_config Settings

The server configuration file is located at `/etc/ssh/sshd_config`. The are:

Setting	Our value	Meaning
Port	22	TCP port sshd listens to
PasswordAuthentication	yes	Allows login with a Unix password, required for the deliberate password-login demonstration.
PermitRootLogin	prohibit-password	Root may not log in with a password over SSH, only with a key (Ubuntu default), reduces brute-force risk on the root account.
PubkeyAuthentication	yes	Enables key-based login.
LoginGraceTime	2m	how long an unauthenticated connection is held open before sshd disconnects it.
StrictModes	Yes	sshd refuses key auth if <code>~/.ssh</code> or <code>authorized_keys</code> have loose permissions (ties directly into your own 700/600 explanation, so it's a natural one-line add).
MaxAuthTries	6	caps failed auth attempts per connection before sshd drops it.

```
$ sudo nano /etc/ssh/sshd_config
```



```

GNU nano 8.7.1 /etc/ssh/sshd_config
# This is the sshd server system-wide configuration file. See
# sshd_config(5) for more information.

# This sshd was compiled with PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games

# The strategy used for options in the default sshd_config shipped with
# OpenSSH is to specify options with their default value where
# possible, but leave them commented. Uncommented options override the
# default value.

# In general, the first uncommented definition of an option takes precedence.
# For options that accept multiple values, like 'Port', subsequent definitions
# are appended to the configuration.

# Note that the above implies that configuration options from snippets in
# /etc/ssh/sshd_config.d/*.conf take precedence over those defined in this
# file. In addition, configuration snippet files are processed in lexical
# order, so options defined in files with names that sort earlier take
# precedence.

# Such configuration snippets may be present in default installations of
# Ubuntu.
include /etc/ssh/sshd_config.d/*.conf

# When systemd socket activation is used (the default), the socket
# configuration must be re-generated after changing Port, AddressFamily, or
# ListenAddress.

# For changes to take effect, run:
#
#   systemctl daemon-reload
#   systemctl restart ssh.socket
#
# Port 22
# AddressFamily any
# ListenAddress 0.0.0.0
# ListenAddress ::

# HostKey /etc/ssh/ssh_host_rsa_key
# HostKey /etc/ssh/ssh_host_ecdsa_key
# HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
# RekeyLimit default none

# Logging
# SyslogFacility AUTH
# LogLevel INFO

# Authentication:

# LoginGraceTime 2m
# PermitRootLogin prohibit-password
# StrictModes yes
# MaxAuthTries 6

```

What this proves: Proves the values documented above are the ACTUAL values in effect on this machine.

3.3 Password-Based Authentication& Failed Attempt Logging

The CLIENT terminal was opened and used to connect to the second account:

```
becca@DESKTOP-DOK0N11:~$ ssh webserver@localhost
webserver@localhost's password: *****
webserver@DESKTOP-DOK0N11:~$ whoami
webserver@DESKTOP-DOK0N11:~$ hostname
webserver@DESKTOP-DOK0N11:~$ pwd
webserver@DESKTOP-DOK0N11:~$ ls
```

The prompt changes from becca@DESKTOP-DOK0N11:~\$ to webserver@DESKTOP-DOK0N11:~\$, and whoami now returns webserver instead of becca, direct proof that this is a genuinely different login session under a different identity, not the same shell relabelled.

```

becca@DESKTOP-DOK0N11:~$ ssh webserver@localhost
Welcome to Ubuntu 26.04 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Sep  7 17:10:23 +04 2026

System load:  0.01               Processes:           62
Usage of /:   0.2% of 1006.85GB  Users logged in:    0
Memory usage: 6%                IPv4 address for eth0: 192.168.199.170
Swap usage:   0%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

   https://ubuntu.com/workshop
Welcome to Ubuntu 26.04 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Sep  7 10:56:44 +04 2026

System load:  0.01               Processes:           57
Usage of /:   0.2% of 1006.85GB  Users logged in:    0
Memory usage: 6%                IPv4 address for eth0: 192.168.199.170
Swap usage:   0%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

   https://ubuntu.com/workshop
Last login: Mon Sep  7 15:34:12 2026 from 127.0.0.1
webserver@DESKTOP-DOK0N11:~$ whoami
webserver
webserver@DESKTOP-DOK0N11:~$ hostname
DESKTOP-DOK0N11
webserver@DESKTOP-DOK0N11:~$ pwd
/home/webserver
webserver@DESKTOP-DOK0N11:~$ ls
test.txt
webserver@DESKTOP-DOK0N11:~$ █

```

What this proves: Demonstrates a working password-based SSH login between the two accounts on the same machine, and that the prompt/identity genuinely changed.

3.4 Deliberately failed login

```

becca@DESKTOP-DOK0N11:~$ ssh webserver@localhost
webserver@localhost's password: (intentionally wrong)

```

Permission denied, please try again.

webserver@localhost's password:

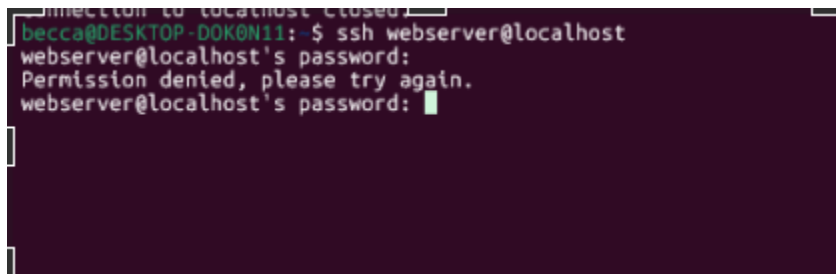
webserver@localhost: Permission denied (publickey, password).

Server-side log evidence for the failed attempt was checked with journalctl or the auth log:

```
$ sudo journalctl -u ssh --since "5 min ago" | grep -i failed
```

```
# or, on systems using the classic auth log:
```

```
$ sudo tail -n 20 /var/log/auth.log
```

A terminal window with a dark purple background. The text shows a user named 'becca' at a machine 'DESKTOP-DOK0N11' attempting to SSH into 'webserver@localhost'. The prompt is '\$ ssh webserver@localhost'. The user enters a password, and the output is 'webserver@localhost's password: Permission denied, please try again.' followed by another prompt 'webserver@localhost's password:'.

```
becca@DESKTOP-DOK0N11:~$ ssh webserver@localhost
webserver@localhost's password:
Permission denied, please try again.
webserver@localhost's password:
```

What this proves: Shows that sshd actively logs and rejects bad credentials, which is the evidence base used again later in the Part F troubleshooting scenario.

3.5 Key-Based Authentication & Key Pair Deployment

```
becca@DESKTOP-DOK0N11:~$ ssh-keygen -t ed25519 -C "teamwebserver-key"
becca@DESKTOP-DOK0N11:~$ ls -l ~/.ssh
becca@DESKTOP-DOK0N11:~$ ssh-copy-id webserver@localhost
becca@DESKTOP-DOK0N11:~$ ssh webserver@localhost
webserver@DESKTOP-DOK0N11:~$ exit
becca@DESKTOP-DOK0N11:~$ sudo cat /home/webserver/.ssh/authorized_keys
becca@DESKTOP-DOK0N11:~$ sudo ls -ld /home/webserver/.ssh
becca@DESKTOP-DOK0N11:~$ ssh -v webserver@localhost
```

Question	Answer
Algorithm chosen	Ed25519: chosen over RSA/ECDSA because it produces short, fast, modern keys with strong security margins and no configurable (and easy to misconfigure) key-length parameter.
Private key file	<code>~/.ssh/id_ed25519</code> : never leaves becca's machine; proves possession without ever being transmitted.
Public key file	<code>~/.ssh/id_ed25519.pub</code> : safe to copy anywhere, including into webserver's <code>authorized_keys</code> ; it can only verify a signature, not create one.
Why the private key never leaves the client	SSH key auth is a challenge-response protocol: the server sends a challenge, the client signs it locally with the private key, and only the signature is sent. The server never needs, and never receives, the private key itself.
Required permissions	<code>~/.ssh</code> must be 700 (owner rwx only) and <code>authorized_keys</code> must be 600 (owner rw only). <code>sshd</code> refuses to trust a key file that is group- or world-writable, since another user could then plant their own key.

```

becca@DESKTOP-DOK0N11:~$ ssh-keygen -t ed25519 -c
Enter file in which the key is (/home/becca/.ssh/id_ed25519):
Old comment: becca@DESKTOP-DOK0N11
New comment:
Comment removed
becca@DESKTOP-DOK0N11:~$ ls -l ~/.ssh
total 24
-rw----- 1 becca becca  0 Sep  7 11:13 authorized_keys
-rw----- 1 becca becca 387 Sep  7 14:51 id_ed25519
-rw-r--r-- 1 becca becca  82 Sep  7 14:51 id_ed25519.pub
-rw----- 1 becca becca 3389 Sep  7 11:08 id_rsa
-rw-r--r-- 1 becca becca  747 Sep  7 11:08 id_rsa.pub
-rw----- 1 becca becca 1120 Sep  7 14:47 known_hosts
-rw----- 1 becca becca  284 Sep  7 14:47 known_hosts.old

```

What this proves: Shows the key pair was generated locally on becca's account and that the two files (private vs public) have different, correct permissions.

```
becca@DESKTOP-DOK0N11:~$ ssh webserver@localhost
Welcome to Ubuntu 26.04 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Sep  7 17:10:23 +04 2026

System load:  0.01               Processes:           62
Usage of /:   0.2% of 1006.85GB  Users logged in:    0
Memory usage: 6%                IPv4 address for eth0: 192.168.199.170
Swap usage:   0%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

https://ubuntu.com/workshop
Welcome to Ubuntu 26.04 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Sep  7 10:56:44 +04 2026

System load:  0.01               Processes:           57
Usage of /:   0.2% of 1006.85GB  Users logged in:    0
Memory usage: 6%                IPv4 address for eth0: 192.168.199.170
Swap usage:   0%

 * Canonical Workshop gives developers fast, composable, reproducible, and
   secure developer environments that are perfect for agentic workflows.

https://ubuntu.com/workshop
Last login: Mon Sep  7 15:34:12 2026 from 127.0.0.1
```

What this proves: A successful login authenticated purely by the key pair, not a password. Below proves that the key is correctly stored in the authorized_keys file.

```
becca@DESKTOP-DOK0N11:~$ sudo cat /home/webserver/.ssh/authorized_keys
[sudo: authenticate] Password:
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIJl1hevUgZLK81sUdxx9THZ+qtnzDR4QwPMUs+JOfoLT
becca@DESKTOP-DOK0N11:~$ █
```

```

becca@DESKTOP-DOK0N11: $ ssh -v webserver@localhost
debug1: OpenSSH_10.2p1 Ubuntu-2ubuntu3.6, OpenSSL 3.5.5 27 Jan 2026
debug1: Reading configuration data /etc/ssh/ssh_config
debug1: Reading configuration data /etc/ssh/ssh_config.d/20-systemd-ssh-proxy.conf
debug1: /etc/ssh/ssh_config line 21: Applying options for *
debug1: Connecting to localhost [127.0.0.1] port 22.
debug1: Connection established.
debug1: loaded pubkey from /home/becca/.ssh/id_rsa: RSA SHA256:m1xb/s/kDnti0cU8WR4sz0+1ML2Mpp09FQ9Ji/v/psI
debug1: identity file /home/becca/.ssh/id_rsa type 0
debug1: no identity pubkey loaded from /home/becca/.ssh/id_rsa
debug1: no pubkey loaded from /home/becca/.ssh/id_ecdsa
debug1: identity file /home/becca/.ssh/id_ecdsa type -1
debug1: no identity pubkey loaded from /home/becca/.ssh/id_ecdsa
debug1: no pubkey loaded from /home/becca/.ssh/id_ecdsa_sk
debug1: identity file /home/becca/.ssh/id_ecdsa_sk type -1
debug1: no identity pubkey loaded from /home/becca/.ssh/id_ecdsa_sk
debug1: loaded pubkey from /home/becca/.ssh/id_ed25519: ED25519 SHA256:igCldRvmPvAUQxQ0y8ULr8Mq8404XLdAXBSCplsEeSM
debug1: identity file /home/becca/.ssh/id_ed25519 type 2
debug1: no identity pubkey loaded from /home/becca/.ssh/id_ed25519
debug1: no pubkey loaded from /home/becca/.ssh/id_ed25519_sk
debug1: identity file /home/becca/.ssh/id_ed25519_sk type -1
debug1: no identity pubkey loaded from /home/becca/.ssh/id_ed25519_sk
debug1: Local version string SSH-2.0-OpenSSH_10.2p1 Ubuntu-2ubuntu3.6
debug1: Remote protocol version 2.0, remote software version OpenSSH_10.2p1 Ubuntu-2ubuntu3.6
debug1: compat_banner: match: OpenSSH_10.2p1 Ubuntu-2ubuntu3.6 pat OpenSSH* compat 0x04000000
debug1: Authenticating to localhost:22 as 'webserver'
debug1: load_hostkeys: fopen /home/becca/.ssh/known_hosts2: No such file or directory
debug1: load_hostkeys: fopen /etc/ssh/ssh_known_hosts: No such file or directory
debug1: load_hostkeys: fopen /etc/ssh/ssh_known_hosts2: No such file or directory
debug1: SSH2_MSG_KEXINIT sent
debug1: SSH2_MSG_KEXINIT received
debug1: kex: algorithm: mlkem768x25519-sha256
debug1: kex: host key algorithm: ssh-ed25519
debug1: kex: server->client cipher: chacha20-poly1305@openssh.com MAC: <implicit> compression: none
debug1: kex: client->server cipher: chacha20-poly1305@openssh.com MAC: <implicit> compression: none
debug1: expecting SSH2_MSG_KEX_ECDH_REPLY
debug1: SSH2_MSG_KEX_ECDH_REPLY received
debug1: Server host key: ssh-ed25519 SHA256:otUXgz0IRKlI1lNkLZp0eU96kk/kqgS5wftB7dLT2To
debug1: load_hostkeys: fopen /home/becca/.ssh/known_hosts2: No such file or directory
debug1: load_hostkeys: fopen /etc/ssh/ssh_known_hosts: No such file or directory

```

What this proves: This exact line in the -v output is the proof that the PUBLIC KEY method authenticated the session, not a fallback to password authentication.

3.6 File Transfer over SSH(SCP and SFTP)

```
becca@DESKTOP-DOK0N11:~$ echo "webserver assignment" > test.txt
becca@DESKTOP-DOK0N11:~$ scp test.txt webserver@localhost:/home/webserver/
becca@DESKTOP-DOK0N11:~$ ssh webserver@localhost
webserver@DESKTOP-DOK0N11:~$ ls -l ~/test.txt
webserver@DESKTOP-DOK0N11:~$ cat ~/test.txt
```

The file was also transferred using an interactive SFTP session, exercising put, get and ls:

```
becca@DESKTOP-DOK0N11:~$ sftp webserver@localhost
sftp> put test.txt
sftp> ls
sftp> get test.txt ./downloaded_test.txt
sftp> bye
```

Source and destination: for scp/put, the source is /home/becca/test.txt on the local (client) side and the destination is /home/webserver/ on the same disk but under a different owner (webserver). Because becca and webserver are different Unix users, the copy only succeeds because SSH first authenticates as webserver and the destination directory (webserver's own home) grants that user write access, the file genuinely crosses a Unix permission boundary even though both accounts sit on one filesystem.

```
becca@DESKTOP-DOK0N11:~$ echo "webserver assignment"> test.txt
becca@DESKTOP-DOK0N11:~$ scp test.txt webserver@localhost:/home/webserver/
test.txt
becca@DESKTOP-DOK0N11:~$ ls -l ~/test.txt
-rw-r--r-- 1 becca becca 21 Sep  7 17:18 /home/becca/test.txt
becca@DESKTOP-DOK0N11:~$ cat ~/test.txt
webserver assignment
becca@DESKTOP-DOK0N11:~$ █
```

What this proves: Confirms the SCP transfer succeeded and that the file is now owned/readable by the receiving account (webserver), not just copied in place.


```
webserver assignment
becca@DESKTOP-DOK0N11:~$ sftp webserver@localhost
Connected to localhost.
sftp> put test.txt
Uploading test.txt to /home/webserver/test.txt
test.txt
sftp> ls
test.txt
sftp> get test.txt ./downloaded_test.txt
Fetching /home/webserver/test.txt to ./downloaded_test.txt
test.txt
sftp> bye
becca@DESKTOP-DOK0N11:~$ █
```

What this proves: Demonstrates the second, interactive transfer method using sftp

4. NGINX Web Server (Part C)

4.1 Installing and Verifying NGINX

```
$ sudo apt install nginx
```

```
$ sudo systemctl status nginx
```

```
becca@DESKTOP-DOK0N11: $ sudo apt install nginx
[sudo: authenticate] Password:
Installing:
  nginx

Installing dependencies:
  nginx-common

Suggested packages:
  fcgiwrap  nginx-doc  ssl-cert

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 153
  Download size: 655 kB
  Space needed: 1860 kB / 1024 GB available

Continue? [Y/n] y
Get:1 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 nginx-common all 1.28.3-2ubuntu1.10 [37.8 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 nginx amd64 1.28.3-2ubuntu1.10 [617 kB]
Fetched 655 kB in 3s (202 kB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 39281 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.28.3-2ubuntu1.10_all.deb ...
Unpacking nginx-common (1.28.3-2ubuntu1.10) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.28.3-2ubuntu1.10_amd64.deb ...
Unpacking nginx (1.28.3-2ubuntu1.10) ...
Setting up nginx-common (1.28.3-2ubuntu1.10) ...
Setting up nginx (1.28.3-2ubuntu1.10) ...
* Upgrading binary nginx
Processing triggers for man-db (2.13.1-1build1) ...
becca@DESKTOP-DOK0N11: $ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-09-07 15:54:26 +04; 14s ago
 Invocation: 2819c2f4bae44dd493c8a169884515c8
    Docs: man:nginx(8)
   Process: 14449 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, >
   Process: 14450 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/>
  Main PID: 14489 (nginx)
    Tasks: 13 (limit: 9328)
   Memory: 10.7M (peak: 22.8M)
      CPU: 130ms
   CGroup: /system.slice/nginx.service
           └─14489 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─14491 "nginx: worker process"
               └─14492 "nginx: worker process"
                 └─14493 "nginx: worker process"
                   └─14494 "nginx: worker process"
                     └─14495 "nginx: worker process"
                       └─14496 "nginx: worker process"
                         └─14497 "nginx: worker process"
                           └─14498 "nginx: worker process"
                             └─14499 "nginx: worker process"
                               └─14500 "nginx: worker process"
                                 └─14501 "nginx: worker process"
                                   └─14502 "nginx: worker process"

Sep 07 15:54:26 DESKTOP-DOK0N11 systemd[1]: Starting nginx.service - A high performance web server and>
Sep 07 15:54:26 DESKTOP-DOK0N11 systemd[1]: Started nginx.service - A high performance web server and>
```

The above confirms NGINX is installed and the master processes are up before any site configuration is attempted.

4.2 NGINX Directory Hierarchy and Symlink Architecture

Path	Purpose	Modified by us?
/etc/nginx/	The central folder where all the NGINX configuration files are found.	Browsed, not edited directly.
/etc/nginx/nginx.conf	master configuration file. it sets global server rules and automatically import individual files like sites-enabled/	Left at distro default (includes conf.d and sites-enabled files).
/etc/nginx/sites-available/	This is the staging area for website configuration files. The sites remain offline until enabled.	Yes: added teamwebserver here.
/etc/nginx/sites-enabled/	Holds only the sites currently served. it contains symlinks pointing to the actual configuration files..	Yes: symlinked teamwebserver here to activate it.
/var/www/	Default web root for website files on Debian/Ubuntu.	Yes: created /var/www/teamwebserver/ and added the content files.
/var/log/nginx/	contains diagnostic logs (access.log and error.log), every request and every startup/config error.	Read only, for evidence of status codes and errors.

The symlink relationship: a file placed in sites-available is offline on its own. This is because NGINX only reads sites-enabled. Creating a symlink (`ln -s /etc/nginx/sites-available/teamwebserver /etc/nginx/sites-enabled/teamwebserver`) is what actually turns the site on, and removing the symlink (not the file) is the standard way to disable a site without deleting its configuration.

```
$ sudo ln -s /etc/nginx/sites-available/teamwebserver /etc/nginx/sites-enabled/teamwebserver
```

```
sudo ls -ld /etc/nginx/ /etc/nginx/nginx.conf /etc/nginx/sites-available/  
/etc/nginx/sites-enabled/ /var/www/ /var/log/nginx/
```

```
becca@DESKTOP-DOK0N11: $ sudo ln -s /etc/nginx/sites-available/teamwebserver /etc/nginx/sites-enabled/teamwebserver
```

```
becca@DESKTOP-DOK0N11: $ ls -ld /etc/nginx/ /etc/nginx/nginx.conf /etc/nginx/sites-available/ /etc/nginx/sites-enabled/ /var/www/ /v  
ar/log/nginx/  
drwxr-xr-x 8 root root 4096 Sep  7 15:54 /etc/nginx/  
-rw-r--r-- 1 root root 1547 Mar 28 01:03 /etc/nginx/nginx.conf  
drwxr-xr-x 2 root root 4096 Sep  7 15:54 /etc/nginx/sites-available/  
drwxr-xr-x 2 root root 4096 Sep  5 23:41 /etc/nginx/sites-enabled/  
drwxr-xr-x 2 root adm 4096 Sep  5 23:29 /var/log/nginx/  
drwxr-xr-x 4 root root 4096 Sep  5 23:36 /var/www/
```

The above shows the symlink relationship between sites-available and sites-enabled described above.

4.3 Custom Website Creation

A site directory was created at /var/www/teamwebserver/ containing website files index.html and 404.html:

```
$ sudo mkdir -p /var/www/teamwebserver  
$ sudo nano /var/www/teamwebserver/index.html  
$ sudo nano /var/www/teamwebserver/404.html
```

index.html identifies the team, the module, and server status, as illustrated below:

```
<!DOCTYPE html>  
<html>  
<head><title>Team Webserver - Web Infrastructure</title></head>  
<body>  
  <h1>Team Webserver</h1>  
  <p>Module: Web Infrastructure</p>
```

```
<p>Members: Edith Amondi, Rebecca Adhiambo, Leatitia Mizero, Nelson Fodjo  
Kamdoum</p>  
<p>Server status: NGINX is running on this machine.</p>  
</body>  
</html>
```



```
GNU nano 8.7.1 /var/www/teamwebserver/index.html  
<!DOCTYPE html>  
<html>  
<head><title>Team Webserver - Web Infrastructure</title></head>  
<body>  
  <h1>Team Webserver</h1>  
  <p>Module: Web Infrastructure</p>  
  <p>Members: Edith Amondi, Rebecca Adhiambo, Leatitia Mizero, Ne  
  <p>Server status: NGINX is running on this machine.</p>  
</body>  
</html>
```

4.4 Server Block Configuration & Syntax validation

The actual site configuration are created at `/etc/nginx/sites-available/teamwebserver`:

```
server {  
    listen 80;  
    server_name localhost;  
    root /var/www/teamwebserver;
```

```

index index.html;

location / {
    try_files $uri $uri/ =404;
}

add_header X-Group-Name "TeamWebserver";

error_page 404 /404.html;
}

```

```

GNU nano 8.7.1 /etc/nginx/sites-available/teamwebserver *
server {
    listen 80;
    server_name localhost;
    root /var/www/teamwebserver;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    add_header X-Group-Name "TeamWebserver";

    error_page 404 /404.html;
    location = /404.html {
        internal;
    }
}

```

Directive	Purpose
listen 80	tells NGINX to listen to incoming traffic from TCP port 80. This is the standard port for encrypted web requests.

<code>server_name localhost</code>	Matches incoming requests whose Host header is 'localhost' to this server block (relevant once more than one site exists).
<code>root /var/www/teamwebserver</code>	The filesystem root directory. NGINX serves files from this site.
<code>index index.html</code>	The default file NGINX returns when a user request targets a directory rather than a specific file.
<code>location / { ... }</code>	A rule block matching all request paths starting at /, eg the whole site.
<code>try_files \$uri \$uri/ =404</code>	Tries the exact file, then the path as a directory, then gives up and returns 404 , this is what makes /doesnotexist correctly fail instead of erroring.
<code>error_page 404 /404.html</code>	Tell NGINX which page to serve when it would otherwise return a bare 404, giving a branded error page instead.

4.5 Validating and Accessing the Website

```
$ sudo nginx -t
$ sudo systemctl restart nginx
$ curl http://localhost
```

```

becca@DESKTOP-DOK0N11:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
becca@DESKTOP-DOK0N11:~$ sudo systemctl restart nginx
becca@DESKTOP-DOK0N11:~$ curl http://localhost/
<!DOCTYPE html>
<html>
<head><title>Team Webserver – Web Infrastructure</title></head>
<body>
  <h1>Team Webserver</h1>
  <p>Module: Web Infrastructure</p>
  <p>Members: Edith Amondi, Rebecca Adhiambo, Leatitia Mizero, Nelson Fodjo Kamdoun</p>
  <p>Server status: NGINX is running on this machine.</p>
</body>
</html>

becca@DESKTOP-DOK0N11:~$ █

```

This shows the Day 3 mandatory checkpoint evidence: the configuration is syntactically valid AND. The custom page is actually being served too.

```

becca@DESKTOP-DOK0N11:~$ curl -i http://localhost/doesnotexist
HTTP/1.1 404 Not Found
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 07 Sep 2026 14:00:52 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive

<html>
<head><title>404 Not Found</title></head>
<body>
  <center><h1>404 Not Found</h1></center>
  <hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>

```

This proves the custom error_page directive is active, not just configured.

5. HTTP Methods, Status Codes, and Custom Headers (Part D)

5.1 Testing HTTP Methods (GET, POST, PUT, DELETE, HEAD)

```
$ curl -X GET http://localhost
$ curl -X POST http://localhost
$ curl -X PUT http://localhost
$ curl -X DELETE http://localhost
$ curl -I http://localhost
```

```
becca@DESKTOP-DOK0N11:~$ curl -X GET http://localhost
<!DOCTYPE html>
<html>
<head><title>Team Webserver – Web Infrastructure</title></head>
<body>
  <h1>Team Webserver</h1>
  <p>Module: Web Infrastructure</p>
  <p>Members: Edith Amondi, Rebecca Adhiambo, Leatitia Mizero, Nelson Fodjo Kamdoun</p>
  <p>Server status: NGINX is running on this machine.</p>
</body>
</html>

becca@DESKTOP-DOK0N11:~$ curl -X POST http://localhost
<html>
<head><title>405 Not Allowed</title></head>
<body>
<center><h1>405 Not Allowed</h1></center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>

becca@DESKTOP-DOK0N11:~$ curl -X PUT http://localhost
<html>
<head><title>405 Not Allowed</title></head>
<body>
<center><h1>405 Not Allowed</h1></center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>

becca@DESKTOP-DOK0N11:~$ curl -X DELETE http://localhost
<html>
<head><title>405 Not Allowed</title></head>
<body>
<center><h1>405 Not Allowed</h1></center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>

becca@DESKTOP-DOK0N11:~$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Tue, 08 Sep 2026 06:17:55 GMT
Content-Type: text/html
Content-Length: 321
Last-Modified: Mon, 07 Sep 2026 13:38:18 GMT
Connection: keep-alive
ETag: "6a9ebe4a-141"
X-Group-Name: TeamWebserver
X-Group-Name: TeamWebserver
Accept-Ranges: bytes

becca@DESKTOP-DOK0N11:~$ █
```

Method	Status returned	Why
GET	200 OK	NGINX's static file handler fully supports GET, the method the site is built for.
HEAD	200 OK (headers only, no body)	HEAD is handled identically to GET internally, minus the response body.
POST	405 Method Not Allowed	A static file server has no handler (no PHP/app backend) to accept a POST body, so NGINX rejects it.
PUT	405 Method Not Allowed	Static serving is read-only from the client's perspective; there is no configured way to write files via HTTP.
DELETE	405 Method Not Allowed	Same reasoning as PUT , no handler exists to remove files based on a client request.

5.2 Deliberate Status Code Generation (200, 403, 404, 405)

Status	How it was produced
200 OK	<code>curl -i http://localhost/</code> , normal, successful request for an existing file.
404 Not Found	<code>curl -i http://localhost/doesnotexist</code> , request for a path that does not exist under the site root.
403 Forbidden	<code>sudo chmod 000 /var/www/teamwebserver/index.html</code> then <code>curl -i http://localhost/</code> , NGINX's worker process (running as www-data)

	can no longer read the file, so it returns 403 instead of serving it.
405 Method Not Allowed	curl -X POST http://localhost , a method the static handler does not support.

```
$ sudo chmod 000 /var/www/teamwebserver/index.html
$ curl -i http://localhost/
$ sudo chmod 644 /var/www/teamwebserver/index.html
$ sudo nginx -t && sudo systemctl reload nginx
$ curl -I http://localhost
```

```

becca@DESKTOP-DOK0N11: $ curl -i http://localhost/
HTTP/1.1 403 Forbidden
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 07 Sep 2026 14:06:09 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>
becca@DESKTOP-DOK0N11: $ sudo chmod 644 /var/www/teamwebserver/index.html
becca@DESKTOP-DOK0N11: $ sudo nginx -t && sudo systemctl reload nginx
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
becca@DESKTOP-DOK0N11: $ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 07 Sep 2026 14:07:20 GMT
Content-Type: text/html
Content-Length: 321
Last-Modified: Mon, 07 Sep 2026 13:38:18 GMT
Connection: keep-alive
ETag: "6a9ebe4a-141"
X-Group-Name: TeamWebserver
Accept-Ranges: bytes

becca@DESKTOP-DOK0N11: $ curl -i http://localhost/nonexistent-file.html
HTTP/1.1 404 Not Found
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 07 Sep 2026 14:10:04 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive

<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>
becca@DESKTOP-DOK0N11: $ curl -i -X POST http://localhost/index.html
HTTP/1.1 405 Not Allowed
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 07 Sep 2026 14:10:20 GMT
Content-Type: text/html
Content-Length: 166
Connection: keep-alive

<html>
<head><title>405 Not Allowed</title></head>
<body>
<center><h1>405 Not Allowed</h1></center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>

```

This proves direct evidence for the four distinct status codes' requirements, each deliberately caused.

```
becca@DESKTOP-DOK0N11: $ sudo nano /etc/nginx/sites-available/teamwebserver
becca@DESKTOP-DOK0N11: $ sudo systemctl reload nginx
becca@DESKTOP-DOK0N11: $ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 07 Sep 2026 14:14:29 GMT
Content-Type: text/html
Content-Length: 321
Last-Modified: Mon, 07 Sep 2026 13:38:18 GMT
Connection: keep-alive
ETag: "6a9ebe4a-141"
X-Group-Name: TeamWebserver
Accept-Ranges: bytes

becca@DESKTOP-DOK0N11: $ sudo nano /etc/nginx/sites-available/teamwebserver
becca@DESKTOP-DOK0N11: $ sudo systemctl reload nginx
becca@DESKTOP-DOK0N11: $ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 07 Sep 2026 14:16:15 GMT
Content-Type: text/html
Content-Length: 321
Last-Modified: Mon, 07 Sep 2026 13:38:18 GMT
Connection: keep-alive
ETag: "6a9ebe4a-141"
X-Group-Name: TeamWebserver
```

This proves the custom response header was actually added to the live configuration and is being returned by the server, not just written in the config file.

6. Telnet, Ports and Raw HTTP (Part E)

Telnet was installed using the command:

```
$ sudo apt install telnet -y
```

6.1 Port Reachability testing (ports 22 and 80)

```
$ telnet localhost 22
```

```
$ telnet localhost 80
```

While port 22 is the registered port for SSH, port 80 is the registered port for plain HTTP.

A successful telnet connection means the TCP three-way handshake has been completed and some process on the machine is actively listening on that port. It says nothing yet about what that process is, only that it exists and accepts connections. Connecting to port 22 immediately returned a banner; SSH-2.0-OpenSSH_10.2p1, which is sshd identifying its protocol version before any authentication begins. This banner is sent in the clear, before encryption is negotiated, which is why it is visible over plain telnet.

```

becca@DESKTOP-DOK0N11: $ sudo apt install telnet -y
Installing:
  telnet

Installing dependencies:
  inetutils-telnet

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 153
  Download size: 111 kB
  Space needed: 319 kB / 1024 GB available

Get:1 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 inetutils-telnet amd64 2:2.7-2ubuntu1.1 [108 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 telnet all 0.17+2.7-2ubuntu1.1 [3650 B]
Fetched 111 kB in 2s (68.4 kB/s)
Selecting previously unselected package inetutils-telnet.
(Reading database ... 39306 files and directories currently installed.)
Preparing to unpack .../inetutils-telnet_2%3a2.7-2ubuntu1.1_amd64.deb ...
Unpacking inetutils-telnet (2:2.7-2ubuntu1.1) ...
Selecting previously unselected package telnet.
Preparing to unpack .../telnet_0.17+2.7-2ubuntu1.1_all.deb ...
Unpacking telnet (0.17+2.7-2ubuntu1.1) ...
Setting up inetutils-telnet (2:2.7-2ubuntu1.1) ...
update-alternatives: using /usr/bin/inetutils-telnet to provide /usr/bin/telnet (telnet) in auto mode
Setting up telnet (0.17+2.7-2ubuntu1.1) ...
Processing triggers for man-db (2.13.1-1build1) ...
becca@DESKTOP-DOK0N11: $ telnet localhost 22
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
SSH-2.0-OpenSSH_10.2p1 Ubuntu-2ubuntu3.6
^C
Connection closed by foreign host.
becca@DESKTOP-DOK0N11: $ telnet localhost 80
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
^CConnection closed by foreign host.
becca@DESKTOP-DOK0N11: $

```

This screenshot confirms that both services are reachable on their expected ports and the raw SSH banner that a browser or ssh client would normally hide.

6.2 Service State Behavior: Running vs. Stopped NGINX

```

$ sudo systemctl status nginx
$ telnet localhost 80
$ sudo systemctl stop nginx
$ telnet localhost 80
$ sudo systemctl start nginx
$ telnet localhost 80

```

Test	Service state	Port	Expected result	Actual result
1	SSH running	22	Connection possible	Connected, SSH banner returned
2	NGINX running	80	Connection possible	Connected, waiting for HTTP request
3	NGINX stopped	80	Connection refused	telnet: Unable to connect to remote host: Connection refused
4	NGINX restarted	80	Connection possible again	Connected, service responsive again

"Connection refused" at the network level means the kernel's TCP stack received the SYN packet but found no process bound to (listening on) port 80, so it immediately replied with a TCP RST rather than leaving the client hanging. This is different from a timeout (nothing listening AND a firewall silently drops the packet), refused is an active, immediate rejection, which is exactly what happens the instant `systemctl stop nginx` tears down its listening socket.

```

becca@DESKTOP-DOK0N11:~$ telnet localhost 80
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
^CConnection closed by foreign host.
becca@DESKTOP-DOK0N11:~$ sudo systemctl stop nginx
becca@DESKTOP-DOK0N11:~$ telnet localhost 80
Trying 127.0.0.1...
telnet: Unable to connect to remote host: Connection refused
becca@DESKTOP-DOK0N11:~$ sudo systemctl start nginx
becca@DESKTOP-DOK0N11:~$ telnet localhost 80
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
^CConnection closed by foreign host.
becca@DESKTOP-DOK0N11:~$ █

```

What this proves: Directly evidences that stopping the service, and only the service, is what changes telnet's result , isolating the service's listening socket as the cause.

6.3 Raw HTTP Request Construction via Telnet/Netcat

```
$ telnet localhost 80
GET / HTTP/1.1
Host: localhost

<-- blank line (press Enter again)
```

```
becca@DESKTOP-DOK0N11: $ telnet localhost 80
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
GET / HTTP/1.1
Host: localhost

HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Tue, 08 Sep 2026 06:30:52 GMT
Content-Type: text/html
Content-Length: 321
Last-Modified: Mon, 07 Sep 2026 13:38:18 GMT
Connection: keep-alive
ETag: "6a9ebe4a-141"
X-Group-Name: TeamWebserver
X-Group-Name: TeamWebserver
Accept-Ranges: bytes

<!DOCTYPE html>
<html>
<head><title>Team Webserver – Web Infrastructure</title></head>
<body>
  <h1>Team Webserver</h1>
  <p>Module: Web Infrastructure</p>
  <p>Members: Edith Amondi, Rebecca Adhiambo, Leatitia Mizero, Nelson Fodjo Kamdoun</p>
  <p>Server status: NGINX is running on this machine.</p>
</body>
</html>
```

The raw response, dissected part by part:

Part	Example content	Meaning
Status line	HTTP/1.1 200 OK	Protocol version (HTTP/1.1), status code (200) and reason phrase (OK) , the very first thing any HTTP response contains.
Headers	Server: nginx/1.x, Content-Type: text/html, Content-Length: 512	Server names the software handling the request; Content-Type tells the client how to interpret the body; Content-Length gives the exact byte count of the body so the client knows when the response ends.
Blank line	(empty line)	The CRLF CRLF sequence that separates headers from the body , its only job is to mark 'headers are finished, body starts now'.
Body	<!DOCTYPE html> ... </html>	The actual HTML of index.html, sent as plain bytes after the blank line.

```

becca@DESKTOP-DOK0N11: $ telnet localhost 80
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
GET / HTTP/1.1
Host: localhost

HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Tue, 08 Sep 2026 06:30:52 GMT
Content-Type: text/html
Content-Length: 321
Last-Modified: Mon, 07 Sep 2026 13:38:18 GMT
Connection: keep-alive
ETag: "6a9ebe4a-141"
X-Group-Name: TeamWebserver
X-Group-Name: TeamWebserver
Accept-Ranges: bytes

<!DOCTYPE html>
<html>
<head><title>Team Webserver – Web Infrastructure</title></head>
<body>
  <h1>Team Webserver</h1>
  <p>Module: Web Infrastructure</p>
  <p>Members: Edith Amondi, Rebecca Adhiambo, Leatitia Mizero, Nelson Fodjo Kamdoun</p>
  <p>Server status: NGINX is running on this machine.</p>
</body>
</html>

```

What this proves: Satisfies the requirement to capture and dissect a full raw HTTP response typed by hand, identifying every one of the four required parts.

Repeat 1: request for a page that does not exist

```

$ telnet localhost 80
GET /nothing.html HTTP/1.1
Host: localhost

```

```
becca@DESKTOP-DOK0N11:~$ telnet localhost 80
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
GET /nothing.html HTTP/1.1
Host: localhost

HTTP/1.1 404 Not Found
Server: nginx/1.28.3 (Ubuntu)
Date: Tue, 08 Sep 2026 06:32:51 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive

<html>
<head><title>404 Not Found</title></head>
<body>
<center><h1>404 Not Found</h1></center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>
```

What this proves: Confirms the same custom error page seen earlier via curl is also returned at the raw protocol level, not only through curl's formatting.

Repeat 2: malformed request (missing Host header)

```
$ telnet localhost 80
GET / HTTP/1.1
```

Because HTTP/1.1 requires a Host header (it is what allows one IP/port to serve multiple named sites), NGINX either returned 400 Bad Request or closed the connection outright, depending on version and configuration strictness.

```
becca@DESKTOP-DOK0N11:~$ telnet localhost 80
Trying 127.0.0.1...
Connected to localhost.
Escape character is '^]'.
GET / HTTP/1.1

HTTP/1.1 400 Bad Request
Server: nginx/1.28.3 (Ubuntu)
Date: Tue, 08 Sep 2026 06:33:24 GMT
Content-Type: text/html
Content-Length: 166
Connection: close

<html>
<head><title>400 Bad Request</title></head>
<body>
<center><h1>400 Bad Request</h1></center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>
Connection closed by foreign host.
becca@DESKTOP-DOK0N11:~$
```

What this proves: Demonstrates that HTTP's plain-text, line-based format lets a client break the protocol contract deliberately and observe the server's real reaction, something a browser normally fixes for you silently.

This technique, typing an HTTP request by hand into telnet, works only because HTTP is unencrypted, human-readable plain text. It would not work against an HTTPS service on port 443, because the very first bytes exchanged there are a TLS handshake (ClientHello/ServerHello, certificate exchange, key negotiation); everything after that, including the HTTP request and response themselves, is encrypted, so raw text typed into a plain telnet session would never be understood by the server and no meaningful response could be read back.

7. API Interaction (Part F)

7.1 Public API Overview (JSONPlaceholder)

API: JSONPlaceholder (<https://jsonplaceholder.typicode.com>).

This is a free public REST API used for testing and prototyping, returning fake but well-structured JSON resources such as posts, comments and users. No authentication or API key is required, which keeps the demonstration free of any credential that would need to be redacted.

7.2 Executing Three Distinct API Requests

#	Method	Endpoint	Purpose
1	GET	/posts/1	Retrieve a single resource by ID , the most basic read operation.
2	GET	/posts?userId=1	Retrieve a filtered collection using a query parameter, returning multiple resources at once.
3	POST	/posts	Create a new resource by sending a JSON body , exercises a write-style request rather than another read.

```
$ curl -i https://jsonplaceholder.typicode.com/posts/1
```

```
$ curl -i "https://jsonplaceholder.typicode.com/posts?userId=1"
```

```
$ curl -i -X POST https://jsonplaceholder.typicode.com/posts \  
-H "Content-Type: application/json" \  
-d '{"title":"Team Webserver","body":"Testing POST","userId":1}'
```

Request	Status	Key JSON fields explained
GET /posts/1	200 OK	id (resource identifier), userId (owning user), title and body (the post content) , a flat single-object response.
GET /posts?userId=1	200 OK	A JSON array of post objects, each with the same shape as above; the query parameter userId filtered the collection server-side.
POST /posts	201 Created	The response echoes back the submitted title/body/userId and adds a server-generated id, confirming (in this sandbox API) that a new resource was accepted.

```

becca@DESKTOP-DOK0N11: $ curl -i https://jsonplaceholder.typicode.com/posts/1
HTTP/2 200
date: Mon, 07 Sep 2026 14:34:14 GMT
content-type: application/json; charset=utf-8
content-length: 292
access-control-allow-credentials: true
cache-control: max-age=43200
etag: W/"124-yiKdLzq05gfBrJFrcdJ8Yq0LGnU"
expires: -1
nel: {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
pragma: no-cache
report-to: {"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=UyhbZ%2F0M01mJoZS7M4Hj8SWBXp3NkwnJgHEYoqocyDE%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1785191026"}],"max_age":3600}
reporting-endpoints: heroku-nel="https://nel.heroku.com/reports?s=UyhbZ%2F0M01mJoZS7M4Hj8SWBXp3NkwnJgHEYoqocyDE%3D&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1785191026"
server: cloudflare
vary: Origin, Accept-Encoding
via: 2.0 heroku-router
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 1000
x-ratelimit-remaining: 999
x-ratelimit-reset: 1785191063
age: 23281
accept-ranges: bytes
cf-cache-status: HIT
cf-ray: a3766ee19d765286-AMS
alt-svc: h3=":443"; ma=86400

{
  "userId": 1,
  "id": 1,
  "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita et cum\nreprehenderit molestiae ut ut quas t
otam\nnostrum rerum est autem sunt rem eveniet architecto"
}becca@DESKTOP-DOK0N11: $ curl -i "https://jsonplaceholder.typicode.com/posts?userId=1"
HTTP/2 200
date: Mon, 07 Sep 2026 14:34:35 GMT
content-type: application/json; charset=utf-8
access-control-allow-credentials: true
cache-control: max-age=43200
etag: W/"aa6-j2NSH739L9uq400ywFMn7Y0C/iY"
expires: -1
nel: {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
pragma: no-cache
report-to: {"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=aExJXDaUpw2S40EsAPcI27Fv5w47CrKebpae0nkVrA%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1785303840"}],"max_age":3600}
reporting-endpoints: heroku-nel="https://nel.heroku.com/reports?s=aExJXDaUpw2S40EsAPcI27Fv5w47CrKebpae0nkVrA%3D&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1785303840"
server: cloudflare
vary: Origin, Accept-Encoding
via: 2.0 heroku-router
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 1000
x-ratelimit-remaining: 999
x-ratelimit-reset: 1785303863

```



```

becca@DESKTOP-DOK0N11: $ curl -i -X POST https://jsonplaceholder.typicode.com/posts -H "Content-Type: application/json" -d '{"title":"Team Webserver","body":"Testing POST","userId":1}'
HTTP/2 201
date: Mon, 07 Sep 2026 14:35:37 GMT
content-type: application/json; charset=utf-8
content-length: 85
location: https://jsonplaceholder.typicode.com/posts/101
access-control-allow-credentials: true
access-control-expose-headers: Location
cache-control: no-cache
etag: W/"55-dM77tBmX9yjAM4ymn/B3LKQCm7k"
expires: -1
nel: {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
pragma: no-cache
report-to: {"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=iBC5Flpgq7hCHknHnQLAwjoFEwd20qDzzLFuR26Xdq0%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1788791737"}],"max_age":3600}
reporting-endpoints: heroku-nel="https://nel.heroku.com/reports?s=iBC5Flpgq7hCHknHnQLAwjoFEwd20qDzzLFuR26Xdq0%3D&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1788791737"
server: cloudflare
vary: Origin, X-HTTP-Method-Override, Accept-Encoding
via: 2.0 heroku-router
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 1000
x-ratelimit-remaining: 999
x-ratelimit-reset: 1788791777
cf-cache-status: DYNAMIC
cf-ray: a37670e759454464-AMS
alt-svc: h3=":443"; ma=86400

{
  "title": "Team Webserver",
  "body": "Testing POST",
  "userId": 1,
  "id": 101
}
becca@DESKTOP-DOK0N11: $

```

What this proves: Evidences three genuinely different requests (different endpoint, different method, different parameters) rather than the same call repeated with a changed value.

7.3 Error Generation, Diagnostics, and Correction

An intentionally incorrect request was made by requesting a resource ID that does not exist:

```
$ curl -i https://jsonplaceholder.typicode.com/posts/99999
```

This returned 404 Not Found with an empty JSON object ({}) as the body, because no post with that ID exists in the API's dataset , the request was syntactically valid but semantically referred to a resource that is not there.

The request was corrected by using a valid ID that is known to exist:

```
$ curl -i https://jsonplaceholder.typicode.com/posts/1
```

```

becca@DESKTOP-DOK0N11:~$ curl -i https://jsonplaceholder.typicode.com/posts/99999
HTTP/2 404
date: Mon, 07 Sep 2026 14:36:47 GMT
content-type: application/json; charset=utf-8
content-length: 2
access-control-allow-credentials: true
cache-control: max-age=43200
etag: W/"2-vyGp6PvFo4RvsFtPoIWeCReyIC8"
expires: -1
nel: {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
pragma: no-cache
report-to: {"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=d1A8KhIGlq31x4qMwYEG0gGQpCU4v8WhWS6nC8MjIfM%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1788790695"}],"max_age":3600}
reporting-endpoints: heroku-nel="https://nel.heroku.com/reports?s=d1A8KhIGlq31x4qMwYEG0gGQpCU4v8WhWS6nC8MjIfM%3D&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1788790695"
server: cloudflare
vary: Origin, Accept-Encoding
via: 2.0 heroku-router
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 1000
x-ratelimit-remaining: 999
x-ratelimit-reset: 1788790697
age: 1112
cf-cache-status: HIT
cf-ray: a376729f0b60fece-AMS
alt-svc: h3=":443"; ma=86400

{}becca@DESKTOP-DOK0N11:~$
becca@DESKTOP-DOK0N11:~$ curl -i https://jsonplaceholder.typicode.com/posts/1
HTTP/2 200
date: Mon, 07 Sep 2026 14:37:03 GMT
content-type: application/json; charset=utf-8
content-length: 292
access-control-allow-credentials: true
cache-control: max-age=43200
etag: W/"124-yiKdLzq05gfBrJFrcdJ8Yq0LGnU"
expires: -1
nel: {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
pragma: no-cache
report-to: {"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=UyhbZ%2F0M01mJoZS7M4Hj8SWBXp

```

What this proves: Shows the complete error-to-correction cycle required by Task E2: cause identified, then a working retry.

8. Environment Troubleshooting Scenario (Part G)

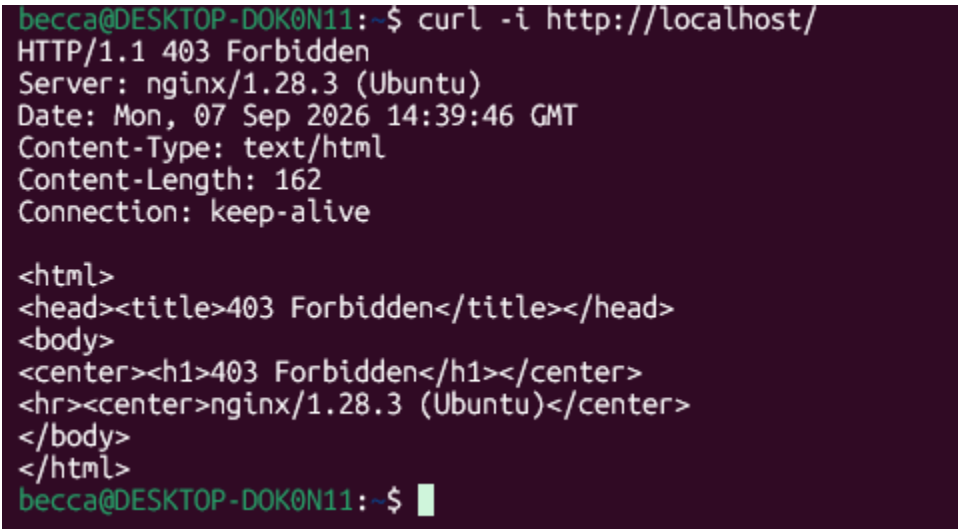
Chosen scenario: Permission problem produces a 403 Forbidden Error on the custom website.

This scenario was deliberately made by removing read permission from index.html, then diagnosed as if the cause was unknown, and finally fixed and verified.

8.1 Stage 1: Problem and Symptom

Visiting localhost, which had been working correctly all week, suddenly outputted '403 Forbidden' instead of the team's homepage. Nothing in the site's HTML or the NGINX site configuration had been changed.

```
$ curl -i http://localhost/
```

A terminal window with a dark purple background. The prompt is 'becca@DESKTOP-DOK0N11:~\$'. The command 'curl -i http://localhost/' has been entered. The output shows an HTTP 403 Forbidden response from nginx/1.28.3 (Ubuntu). The response includes headers for Date, Content-Type, Content-Length, and Connection. The body contains an HTML document with a title '403 Forbidden', a heading '403 Forbidden', and a line identifying the server as 'nginx/1.28.3 (Ubuntu)'.

```
becca@DESKTOP-DOK0N11:~$ curl -i http://localhost/
HTTP/1.1 403 Forbidden
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 07 Sep 2026 14:39:46 GMT
Content-Type: text/html
Content-Length: 162
Connection: keep-alive

<html>
<head><title>403 Forbidden</title></head>
<body>
<center><h1>403 Forbidden</h1></center>
<hr><center>nginx/1.28.3 (Ubuntu)</center>
</body>
</html>
becca@DESKTOP-DOK0N11:~$
```

The above captures the symptom exactly as a system administrator would first observe it, before any investigation begins.

8.2 Stage 2: Log and System Evidence

The NGINX error log was checked first, since 403s are almost always logged with a specific reason:

```
$ sudo tail -n 20 /var/log/nginx/error.log
```

The log contained a line: 'Permission denied'(line 3) while reading the path /var/www/teamwebserver/index.html, immediately pointing at a file system permission issue rather than a configuration or DNS problem.

```
becca@DESKTOP-DOK0N11: $ sudo tail -n 20 /var/log/nginx/error.log
2026/09/05 23:29:25 [notice] 5487#5487: using inherited sockets from "5;6;"
2026/09/07 15:54:26 [notice] 14489#14489: using inherited sockets from "5;"
2026/09/07 18:00:52 [error] 16083#16083: *2 open() "/var/www/teamwebserver/404.html" failed (2: No such file or directory), client: 127.0.0.1, server: localhost, request: "GET /doesnotexist HTTP/1.1", host: "localhost"
2026/09/07 18:06:09 [error] 16081#16081: *3 open() "/var/www/teamwebserver/index.html" failed (13: Permission denied), client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", host: "localhost"
2026/09/07 18:10:04 [error] 16196#16196: *5 open() "/var/www/teamwebserver/404.html" failed (2: No such file or directory), client: 127.0.0.1, server: localhost, request: "GET /nonexistent-file.html HTTP/1.1", host: "localhost"
2026/09/07 18:30:49 [error] 16637#16637: *4 open() "/var/www/teamwebserver/404.html" failed (2: No such file or directory), client: 127.0.0.1, server: localhost, request: "GET /nothing.html HTTP/1.1", host: "localhost"
2026/09/07 18:39:46 [error] 16644#16644: *7 open() "/var/www/teamwebserver/index.html" failed (13: Permission denied), client: 127.0.0.1, server: localhost, request: "GET / HTTP/1.1", host: "localhost"
```

What this proves: This narrowed the investigation from 'the site is broken' to 'the worker process cannot read this specific file'.

8.3 Stage 3: Technical Investigation

The investigation followed the evidence from the log directly to the file's permissions:

```
$ ls -l /var/www/teamwebserver/index.html
$ sudo nginx -t
$ ps aux | grep nginx
```

ls -l showed index.html with permission bits ----- (000) , unreadable by anyone, including the owner.

sudo nginx -t confirmed the configuration itself was syntactically valid, ruling out a config-syntax cause.

ps aux | grep nginx confirmed the worker processes run as www-data, which needs at least read (r) permission on the file to serve it , and 000 denies read access to every user, including www-data.

```

becca@DESKTOP-DOK0N11: $ ls -l /var/www/teamwebserver/index.html
----- 1 root root 321 Sep  7 17:38 /var/www/teamwebserver/index.html
becca@DESKTOP-DOK0N11: $ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
becca@DESKTOP-DOK0N11: $ ps aux | grep nginx
root      16632  0.0  0.0 14848 2292 ?        Ss   18:27   0:00 nginx: master process /usr/sbin/nginx -g daemon
on; master_process on;
www-data  16633  0.0  0.0 16820 5676 ?        S    18:27   0:00 nginx: worker process
www-data  16634  0.0  0.0 16820 5676 ?        S    18:27   0:00 nginx: worker process
www-data  16635  0.0  0.0 16820 5348 ?        S    18:27   0:00 nginx: worker process
www-data  16636  0.0  0.0 16820 5864 ?        S    18:27   0:00 nginx: worker process
www-data  16637  0.0  0.0 16820 5864 ?        S    18:27   0:00 nginx: worker process
www-data  16638  0.0  0.0 16820 5676 ?        S    18:27   0:00 nginx: worker process
www-data  16639  0.0  0.0 16820 5348 ?        S    18:27   0:00 nginx: worker process
www-data  16640  0.0  0.0 16820 5348 ?        S    18:27   0:00 nginx: worker process
www-data  16641  0.0  0.0 16820 5864 ?        S    18:27   0:00 nginx: worker process
www-data  16643  0.0  0.0 16820 5348 ?        S    18:27   0:00 nginx: worker process
www-data  16644  0.0  0.0 16820 5864 ?        S    18:27   0:00 nginx: worker process
www-data  16645  0.0  0.0 16820 5284 ?        S    18:27   0:00 nginx: worker process
becca    16973  0.0  0.0   4128 2424 pts/7    S+   18:42   0:00 grep --color=auto nginx
becca@DESKTOP-DOK0N11: $

```

What this proves: Shows the reasoning step that isolated the cause to file permissions specifically, rather than the site configuration.

8.4 Stage 4: Root Cause Analysis

Root cause: index.html had its permission bits set to 000 (no read/write/execute for owner, group, or others) , most likely left over from the 403 test performed earlier in Task C6, where chmod 000 was applied and not correctly reverted.

8.5 Stage 5: Remediation and Verification

```

$ sudo chmod 644 /var/www/teamwebserver/index.html
$ ls -l /var/www/teamwebserver/index.html
$ curl -i http://localhost/

```

chmod 644 restores standard file permissions: read/write for the owner, and read-only for group and others , enough for the www-data worker process to read and serve the file, without granting it write access.

```

becca@DESKTOP-DOK0N11: $ sudo chmod 644 /var/www/teamwebserver/index.html
becca@DESKTOP-DOK0N11: $ ls -l /var/www/teamwebserver/index.html
-rw-r--r-- 1 root root 321 Sep  7 17:38 /var/www/teamwebserver/index.html
becca@DESKTOP-DOK0N11: $ curl -i http://localhost/
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 07 Sep 2026 14:42:56 GMT
Content-Type: text/html
Content-Length: 321
Last-Modified: Mon, 07 Sep 2026 13:38:18 GMT
Connection: keep-alive
ETag: "6a9ebe4a-141"
X-Group-Name: TeamWebserver
X-Group-Name: TeamWebserver
Accept-Ranges: bytes

<!DOCTYPE html>
<html>
<head><title>Team Webserver – Web Infrastructure</title></head>
<body>
  <h1>Team Webserver</h1>
  <p>Module: Web Infrastructure</p>
  <p>Members: Edith Amondi, Rebecca Adhiambo, Leatitia Mizero, Nelson Fodjo Kamdoum</p>
  <p>Server status: NGINX is running on this machine.</p>
</body>
</html>

becca@DESKTOP-DOK0N11: $ curl -i http://localhost/

```

What this proves: Final proof that the diagnosed cause was correct: fixing exactly that one permission, and nothing else, restored the site to working order.

9. Conclusion

From parts A-F, the web infrastructure was built and initialized from one machine, each subsequent process depending on the one before it. User accounts ; **becca** and **webserver**; provide the identities that SSH authenticates. SSH provides a secure administrative channel; first with a password, then with a key pair whose private half never leaves the client. The same channel was used to move files across a real Unix permission boundary with SCP and SFTP.

NGINX turns port 80 into a service: the files under `/var/www/teamwebserver/` are inactive until the site configuration in `sites-available` is symlinked into `sites-enabled` and validated with `nginx -t`, at which point `curl` can retrieve them over HTTP. Telnet was used to strip away everything `curl` normally does automatically , TLS, formatting, retries and show that HTTP is, underneath, just lines of plain text terminated by CRLF CRLF; stopping and restarting NGINX proved that a 'listening service' is a specific socket that exists only while the process is running, which is also why 'connection refused' is a meaningful, precise error rather than a generic system failure.

The public API work showed the same request/response model, method, endpoint, status codes, headers, body , applying identically to a remote JSON API as it does to a local static site, just with a different host and a structured body instead of HTML. Finally, the troubleshooting exercise tied the whole system together: a single misconfigured permission bit was traced from symptom (403 in the browser/curl), through evidence (permission denied), through investigation (confirming via `ls -l` and `ps aux` which process needed which access), to a one-line fix , the same log-first, narrow-down methodology used informally throughout every other part of this assignment.

Together, these six parts demonstrate that 'client' and 'server' are roles defined by behaviour (who listens, who initiates), not by hardware , and that ports, permissions, protocols and processes are the concrete mechanisms that make those roles work, whether the client and server sit on two continents or, as in this assignment, inside the very same Linux kernel.

10. References & Submission Directory Structure

1. Canonical Ltd. (2024). *Ubuntu server documentation: OpenSSH*. Ubuntu Documentation. <https://ubuntu.com/server/docs/openssh-server>
2. Canonical Ltd. (2024). *Ubuntu documentation: NGINX web server*. Ubuntu Documentation. <https://ubuntu.com/server/docs/web-servers-nginx>
3. curl. (2024). *curl manual*. <https://curl.se/docs/manual.html>
4. Fielding, R., & Reschke, J. (2014). *Hypertext Transfer Protocol (HTTP/1.1): Message syntax and routing* (RFC 7230). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7230>
5. NGINX Inc. (2024). *NGINX documentation*. <https://nginx.org/en/docs/>
6. OpenBSD Project. (2024). *sshd_config(5) manual page*. OpenSSH. https://man.openbsd.org/sshd_config
7. Postel, J., & Reynolds, J. (1983). *Telnet protocol specification* (RFC 854). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc854>
8. Typicode. (2024). *JSONPlaceholder: Free fake API for testing and prototyping*. <https://jsonplaceholder.typicode.com/>
9. OpenSSH. (2025). *OpenSSH 10.2 release notes*. OpenBSD Project. <https://www.openssh.org/releases/notes.html>
10. OpenSSH. (n.d.). *ssh_config(5) - OpenSSH SSH client configuration file*. OpenBSD Manual Pages. https://man.openbsd.org/ssh_config.5
11. Ylonen, T., & Lonvick, C. (2006). *The Secure Shell (SSH) Authentication Protocol* (RFC 4252). Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc4252>
12. Barrett, D. J., Silverman, R. E., & Byrnes, R. G. (2005). *SSH, The Secure Shell: The Definitive Guide* (2nd ed.). O'Reilly Media.
13. Nemeth, E., Snyder, G., Hein, T. R., Whaley, B., & Mackin, D. (2017). *UNIX and Linux System Administration Handbook* (5th ed.). Addison-Wesley Professional.

10.1 Sources, Credits & AI Transparency Log

OpenAI. (2026). *ChatGPT* (Sep 2026 version) [Large language model].
<https://chat.openai.com/chat>

Used to: [e.g., "clarify the meaning of the 'Connection refused' vs 'timeout' distinction at the TCP level before we write section 6.2"]. Date: [8th September 2026]. Team member: [Nelson Fodjo].

Reply Obtained: A 'Connection refused' error occurs at the transport layer when the client sends a TCP SYN packet to a target port, but the host receives it and actively sends back a TCP RST (reset) packet. This means the network path and host are reachable, but no service is listening on that specific port.

In contrast, a 'Connection timeout' means the client sent the initial TCP SYN packet, but received no response at all (neither a SYN-ACK nor an RST) before the timer expired. This usually indicates that a firewall along the path or on the host dropped the packet silently (a DROP or DROP-like rule), or that the target IP address itself is entirely unreachable or down.

For Section 6.2, make sure to highlight that when NGINX is stopped, Telnet returns 'Connection refused' because port 80 is no longer listening, but the local TCP/IP stack is still active and immediately drops the handshake attempt with an RST.

We also used the same model for a final review to make sure that all the boxes were checked and the technical documentation covered all sectors in the rubric.

10.2 Link to submission folder

Attached is the link to the github repository with the submission folder containing our configuration files, website files, evidence screenshots and PUBLIC SSH key.

<https://github.com/Becca4375/WEBSERVER.git>